

Mesh Dependence in PDE-Constrained Optimisation with Firedrake, pyadjoint, and PETSc TAO

Reproducing Schwedes’ iteration-count benchmark and extending it to the inner Krylov solve

Bert Jim Chang
Supervisor: David A. Ham
Department of Mathematics

Context

Firedrake is an open-source finite element library, and pyadjoint is its adjoint extension for optimisation. Together they let a user write a PDE-constrained optimisation problem in near-mathematical syntax and have it solved automatically. The library currently supports several optimisation backends: SciPy for general use, ROL as the backend for the Fireshape shape-optimisation toolkit, and PETSc TAO as a more recent option that recent Firedrake demos point at for parallel workloads. For this direction to hold up, TAO has to be shown to behave correctly on the kind of problem Firedrake users actually solve: one where the control is a *function* on a domain rather than a vector of numbers, and where the natural way to measure angle and distance on the control space is a Hilbert-space inner product rather than the standard Euclidean one.

The mesh-dependence phenomenon

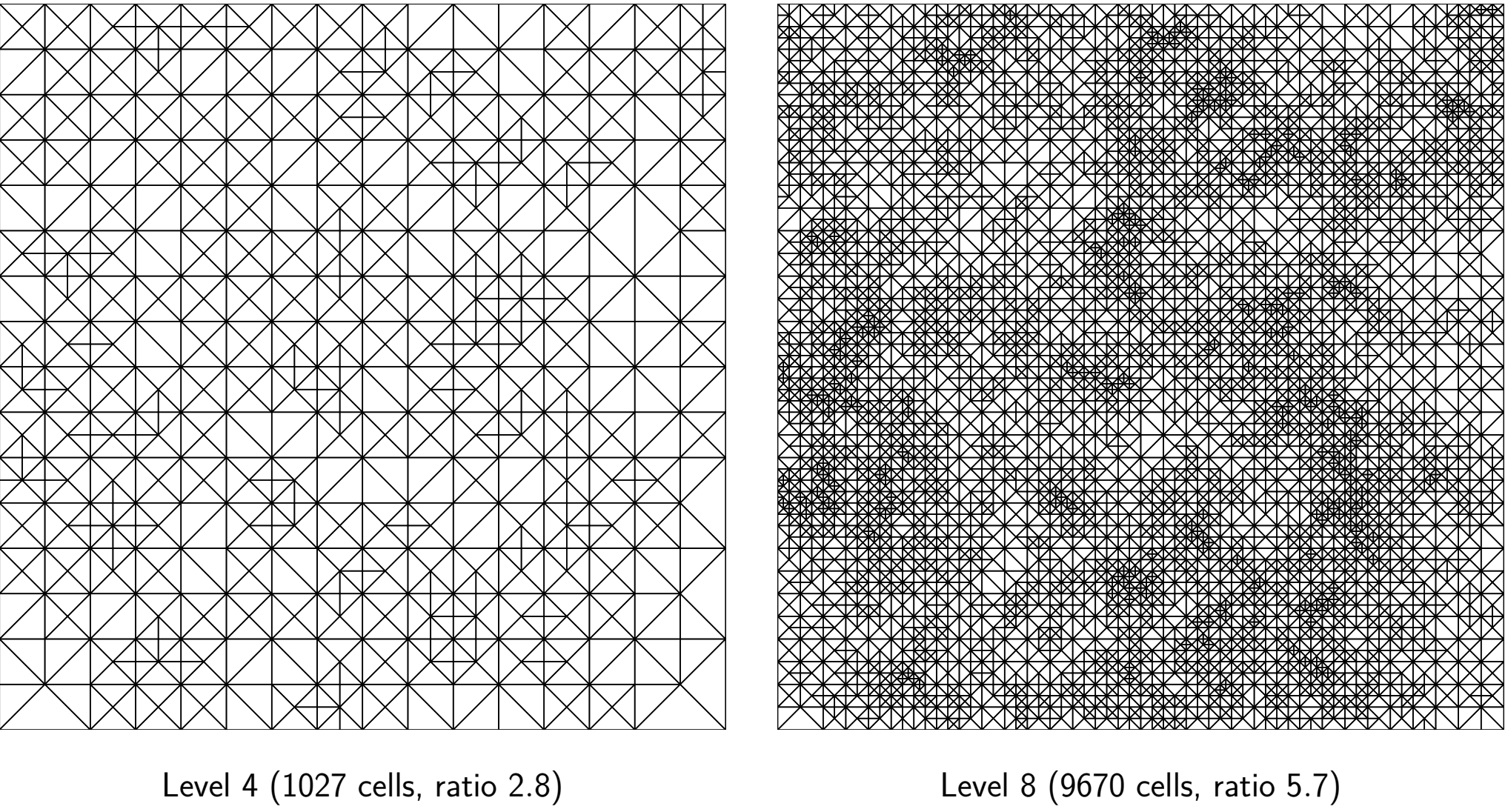
The **finite element method** approximates a function on a domain by triangulating the domain (splitting it into a *mesh*) and representing the function as a low-degree polynomial on each triangle. Users need finer triangles wherever the solution varies rapidly, so meshes in real applications are almost never uniform. Schwedes et al. (2017) show that gradient-based optimisation over a Hilbert function space is mesh-independent if and only if the algorithm respects that space’s inner product. Their *iteration-count benchmark* makes the effect empirically visible: the same Poisson optimal control problem is solved on a sequence of increasingly non-uniformly refined meshes, and the number of outer iterations each optimiser takes to converge is recorded. Optimisers hard-coded to the Euclidean ℓ^2 inner product produce iteration counts that grow polynomially with mesh refinement, while ones using the natural L^2 inner product stay flat. The fix runs through a single operator, the *Riesz map*. It converts a functional’s derivative (which naturally sits in the dual space H^*) into an element of H that the optimiser can use as a search direction. In the discrete L^2 setting,

$$\mathcal{R}_{L^2} = M_h^{-1},$$

the inverse of the assembled Galerkin mass matrix. Skipping this step leaves the mesh-scaling factor in M_h uncanceled, which produces the polynomial growth on non-uniformly refined meshes.

Test problem and setup

Poisson optimal control: find a source term $m \in L^2(\Omega)$ that minimises
$$\tilde{J}(m) = \frac{1}{2} \|u - d\|_{L^2(\Omega)}^2 + \frac{\alpha}{2} \|m\|_{L^2(\Omega)}^2, \quad \text{s.t.} \quad -\Delta u = m \text{ on } \Omega, \quad u = 0 \text{ on } \partial\Omega,$$
 on $\Omega = (0, 1)^2$ with desired state $d = \sin(\pi x) \sin(\pi y)$ and regularisation $\alpha = 10^{-4}$. The reduced gradient is derived by hand using the adjoint method and computed at run time using pyadjoint, which records a tape of the forward solve and plays it back in reverse. Verification is by a manufactured-solution convergence test and by a Taylor test on the reduced gradient, both showing second-order behaviour. The mesh sequence follows Schwedes et al. (2017, Sect. 2.3.2.1) exactly: starting from a uniform 8×8 triangulation of the unit square, at each level every cell is independently marked with probability $p = 0.35$ and Rivara-bisected on its longest edge, with LEPP chain propagation keeping the mesh conforming. Refinement scatters through the interior, so the Dirichlet boundary is treated symmetrically. Realised $h_{\max}/h_{\min}$ grows from 1 to ~ 5.7 across levels 0–10.



Objectives

- 1. **Show that TAO works correctly at the outer level.** Reproduce Schwedes’ iteration-count benchmark in the Firedrake + pyadjoint + TAO stack. Find the settings that make TAO’s `lmvm` solver match the flat row. Test the other TAO solvers (`nls`, `bqnls`, `cg`) to see which inherit the good behaviour and which do not.
- 2. **Fix the mesh-dependence at the inner solver level.** TAO’s `nls` solver takes only two outer Newton iterations because the Poisson problem is quadratic. Each Newton step, however, solves a linear system with an inner CG whose iteration count the outer table does not show. Show that this inner count grows with mesh grading and that an L^2 Riesz-map preconditioner fixes it. This is the piece Schwedes does not test.
- 3. **Work out the pyadjoint interface.** Determine how the `TAOSolver` wrapper should expose the Riesz-map mechanism so end users get correct mesh-independent behaviour by default.

Reproducing the experiment (outer L-BFGS)

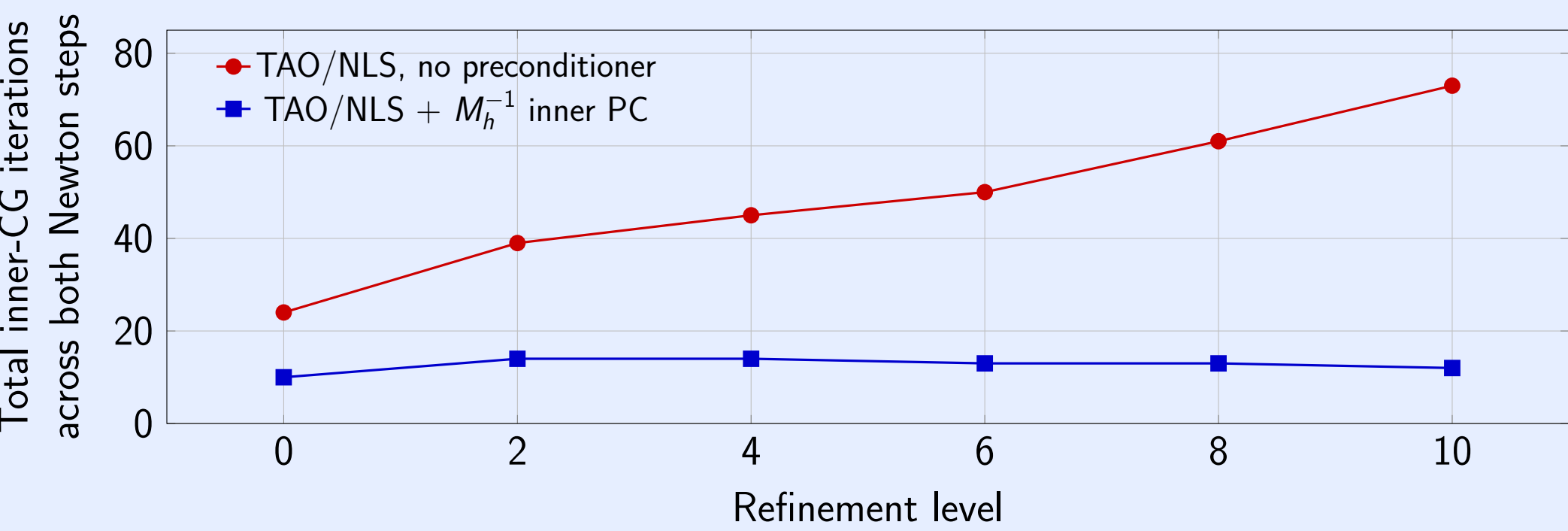
Both L-BFGS optimisers were run to $\|\mathcal{R}_{L^2}(J')\|_{L^2(\Omega)} \leq 10^{-7}$ across six refinement levels. The reference implementation is a hand-written Hilbert-space L-BFGS following Schwedes et al. (2017, Sect. 1.5.4.1); the SciPy row is L-BFGS-B in ℓ^2 with the convergence check done externally in L^2 .

Optimiser (inner product used)	L=0	L=2	L=4	L=6	L=8	L=10
SciPy L-BFGS-B (ℓ^2)	13	31	35	47	47	55
Hand-written Hilbert L-BFGS (L^2)	6	8	6	6	5	4

Cell counts across the sweep: 128, 340, 1027, 3174, 9670, 29474. Realised $h_{\max}/h_{\min}$: 1.00, 2.00, 2.83, 4.00, 5.66, 5.66. The SciPy ℓ^2 row grows roughly $4\times$ across the sweep; the Hilbert L^2 row stays flat between 4 and 8 iterations. TAO’s `lmvm` reproduces the flat row once its default history length of five is raised to 100.

New result: preconditioning the inner CG of TAO/NLS

TAO/NLS converges in two outer Newton steps at every mesh (the Poisson problem is quadratic in m). The total *inner-CG* count across those two steps, however, grows with refinement. Attaching an L^2 Riesz-map preconditioner to the inner KSP, using a plain PETSc Python PC (`RieszMapPCContext`) that assembles M_h once and applies M_h^{-1} on every inner call, restores mesh-independence.



Baseline inner-CG counts grow from 24 to 73 across the sweep (levels 0 to 10, up to 29474 cells). With the L^2 Riesz-map preconditioner installed on TAO’s inner KSP, the count stays between 10 and 14 across the whole sweep. This is the outer-optimiser argument carried down to the inner Krylov solve, and it is what Schwedes et al. (2017, Sect. 1.3.3) points at via Kirby (2010) but does not test experimentally.

Work plan for the remaining time

- **Diagnose bqnls’s line search.** Rerun the mesh sweep with `tao_ls_monitor` to distinguish a history-size issue (as with stock `lmvm`) from a projected line-search bug.
- **H^1 Riesz preconditioner sweep.** The preconditioner module already handles the H^1 form. Adding an H^1 column to the inner-CG table would round off the story symmetrically with Schwedes’ H^1 variant of the benchmark.
- **Interface design writeup.** Draft a short document laying out how the `TAOSolver` wrapper should expose the Riesz-map mechanism to users.

References

- Schwedes, Ham, Funke, Piggott (2017). *Mesh Dependence in PDE-Constrained Optimisation*. SpringerBriefs.
- Kirby (2010). From functional analysis to iterative methods. *SIAM Review* 52(2).
- Rathgeber et al. (2016). Firedrake: automating the finite element method. *ACM TOMS* 43(3).
- Mitusch, Funke, Dokken (2019). dolfin-adjoint 2018.1. *JOSS* 4(38).
- Schöberl (1997). NETGEN: an advancing-front 2D/3D mesh generator. *Comput. Vis. Sci.* 1(1).